

AI TECH - Akademia Innowacyjnych Zastosowań Technologii Cyfrowych. Programu Operacyjnego Polska Cyfrowa na lata 2014-2020

SKRYPT DO LABORATORIUM

Uczenie głebokie

LABORATORIUM 1:

Uczenie sieci splotowych

Natalia Głowacka



Projekt współfinansowany ze środków Unii Europejskiej w ramach Europejskiego Funduszu Rozwoju Regionalnego Program Operacyjny Polska Cyfrowa na lata 2014-2020, Oś Priorytetowa nr 3 "Cyfrowe kompetencje społeczeństwa" Działanie nr 3.2 "Innowacyjne rozwiązania na rzecz aktywizacji cyfrowej"
Tytuł projektu: „Akademia Innowacyjnych Zastosowań Technologii Cyfrowych (AI Tech)”

---

1. Opis ćwiczenia

Wymagania wstępne:

Wymagania w odniesieniu do studenta:

Właściwe przygotowanie się studenta do zajęć pozwoli na osiągnięcie celów ćwiczenia. Przed przystąpieniem do ćwiczenia student powinien:

- powtórzyć wiedzę nabytą w czasie wykładów,
- zapoznać się z instrukcją do ćwiczenia ilustrującą podstawowe zagadnienia z zakresu tematyki ćwiczenia.

Dodatkowo wymagana jest od studenta:

- podstawowa umiejętność programowania,
- podstawowa znajomość języka Python.

Wymagania w odniesieniu do stanowiska laboratoryjnego:

Stanowisko laboratoryjne powinno być wyposażone w komputer z dostępem do sieci komputerowej oraz z następującymi zasobami:

- przykładowe kody programów (załącznik),
- Python 3,
- pakiety NumPy, Matplotlib, TensorFlow, Keras, Seaborn,
- Jupyter Notebook,
- inne: przeglądarka WWW.

Cele ćwiczenia:

Celem ćwiczenia jest praktyczne przedstawienie wiedzy zdobytej podczas wykładów. Podczas realizacji ćwiczenia laboratoryjnego studenci zdobędą wiedzę i umiejętności w zakresie podstawowych zagadnień dotyczących sieci splotowych, operacji splotu, wykorzystania podstawowych warstw sieci splotowych oraz implementacji i trenowania prostej sieci splotowej.

Spodziewane efekty kształcenia - umiejętności i kompetencje:

Po zakończeniu ćwiczenia laboratoryjnego student będzie posiadał umiejętność w zakresie obliczania wyniku operacji splotu, a także obliczania rozmiaru modelu wyjściowego po zastosowaniu kilku warstw charakterystycznych dla sieci splotowych. Zdobędzie także podstawową wiedzę w zakresie implementacji i trenowania prostej sieci splotowej.

Metody dydaktyczne:

Na początku student realizuje zadania przykładowe, które prezentują poszczególne elementy obliczania wyniku operacji splotu dla przykładowych danych. W kolejnym kroku realizuje zadania według wytycznych, na podstawie zdobytych umiejętności i wiedzy. Kody źródłowe i wyniki wykonywanych programów student powinien umieszczać w dokumencie elektronicznym, który stanie się sprawozdaniem z ćwiczenia laboratoryjnego.

Materiały wprowadzające i pomocnicze:

- Numpy - dokumentacja biblioteki https://numpy.org/doc/,
- Matplotlib - dokumentacja biblioteki https://matplotlib.org/stable/contents.html,
- Tensorflow – dokumentacja biblioteki https://www.tensorflow.org/api_docs,
- Keras - dokumentacja biblioteki https://keras.io/api/,
- Seaborn - dokumentacja biblioteki https://seaborn.pydata.org,
- Python 3 - dokumentacja języka https://docs.python.org/3/.

Zasady oceniania/warunek zaliczenia ćwiczenia:

Każde z realizowanych podczas ćwiczenia laboratoryjnego zadań będzie podlegało ocenie. W trakcie realizacji ćwiczenia, mogą zostać przydzielone dodatkowe zadania do realizacji. Maksymalna liczba punktów do zdobycia wynosi 10.

Wykaz literatury podstawowej do ćwiczenia:

1. Treści wykładowe do przedmiotu “Uczenie głębokie”
2. Bengio Yoshua, Courville Aaron, Goodfellow Ian, Deep Learning, Systemy uczące się, PWN 2018
3. Andrew W. Trask, Zrozumieć głębokie uczenie, PWN, 2019

2. Przebieg ćwiczenia

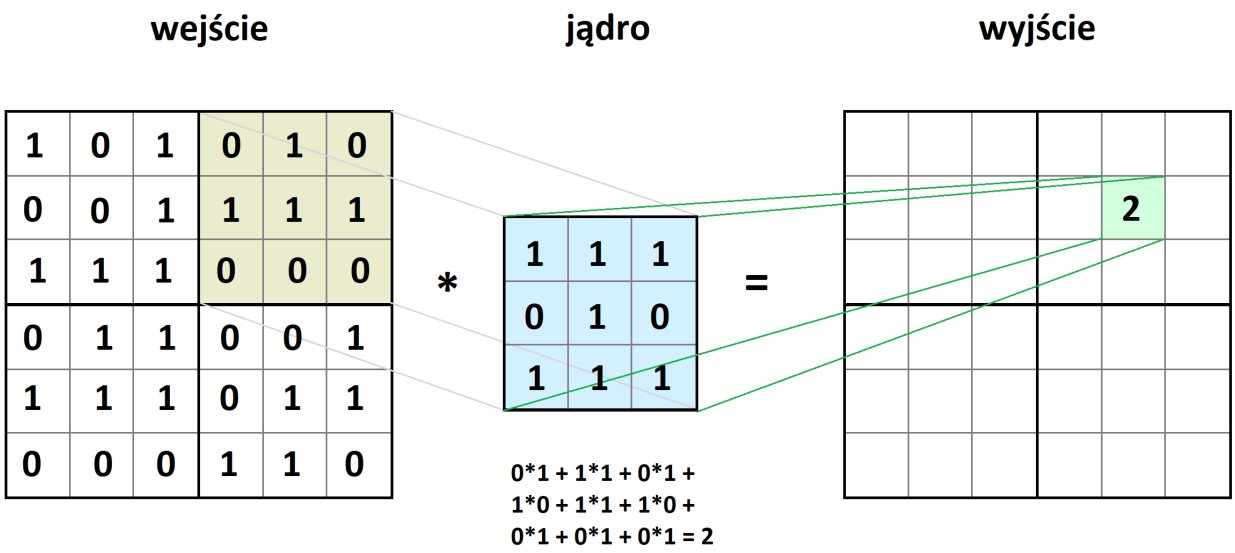
1. Zapoznanie się z instrukcją laboratoryjną (przed ćwiczeniem)
2. Zadanie 1 - Obliczenie splotu ręcznie (30 min.)
3. Zadanie 2 - Wykorzystanie splotu różnych jąder na obrazach (25 min.)
4. Zadanie 3 - Obliczanie rozmiaru wyjściowego modelu (30 min.)
5. Zadanie 4 - Implementacja i trenowanie prostej sieci splotowej (40 min.)
6. Przesłanie sprawozdania z zajęć (10 min.)

3. Wprowadzenie do ćwiczenia

Zadanie 1. Obliczenie splotu ręcznie

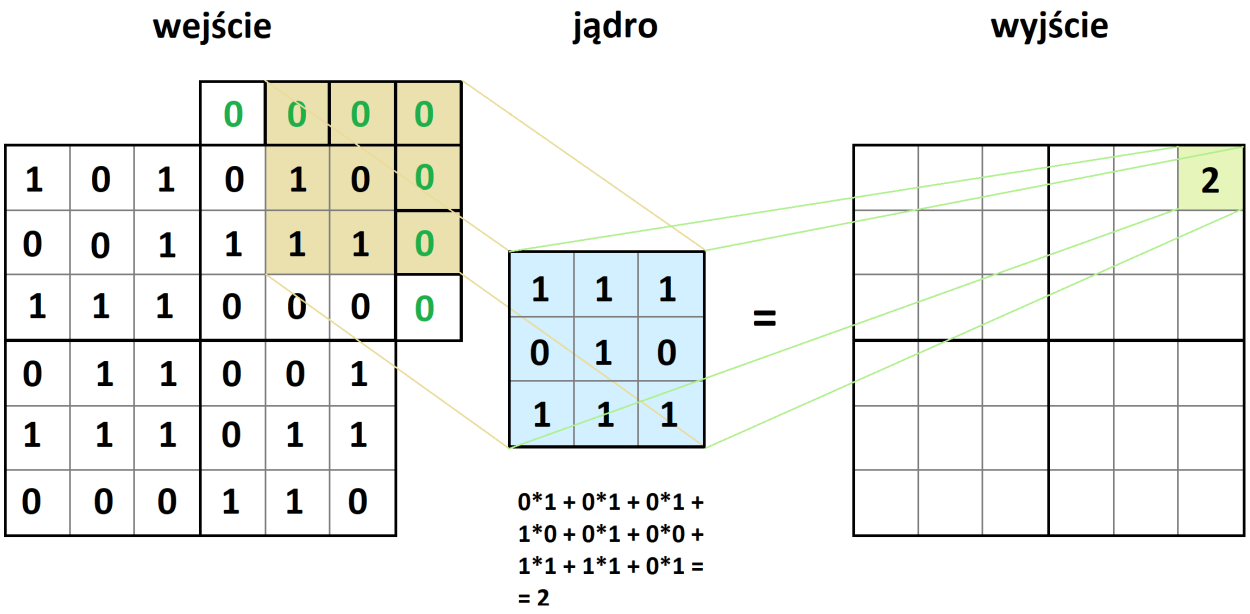
Podstawą splotu jest wykorzystanie faktu, iż każde z wejść jest dodawane do swoich lokalnych sąsiadów (w ramach lokalnego pola recepcyjnego) z pewną wagą wyrażoną poprzez jądro.

Na rysunku poniżej przedstawiony został przykład obliczenia splotu dla przykładowych wartości wejścia oraz jądra.



Splot możemy obliczyć wtedy, kiedy środek jądra może zostać nałożony na pole w całości. Dla przypadku przedstawionego na powyższym rysunku (obraz 6x6), jądro o wymiarach 3x3 możemy nałożyć na 16 pól, dlatego wyjściowa mapa cech miałaby wymiar 4x4. Jeśli chcemy uzyskać mapę cech o takim samym wymiarze jak wejście, należy wykorzystać *technikę dopełniania*. W przypadku obliczenia wartości splotu na krawędziach, często dodaje się zera wokół danych wejściowych (*zero padding*). Na rysunku poniżej przedstawiony został przykład zastosowania uzupełniania zerami do obliczenia splotu

dla wartości położonej na krawędzi obrazu wejściowego.



1a. Wykorzystując zdefiniowane poniżej jądra oraz obraz wejściowy, oblicz wartości wyjściowe dla każdej z wartości wejściowych (wyjście jest takiego samego rozmiaru jak wejście - zastosuj uzupełnianie zerami).

```
import os
from pathlib import Path
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'last_expr'
# Lokalne katalogi cache, wspólne dla uruchomienia z Jupyter i VS Code.
os.environ['KERAS_HOME'] = str(Path('.keras').resolve())
os.environ['MPLCONFIGDIR'] = str(Path('.mplconfig').resolve())
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
```

```
import numpy as np
kernels = {
    "k1":   np.array([
                            [2., 0., -2.],
                            [2., 0., -2.],
                            [2., 0., -2.],
                            ]),
    "k2":  np.array([
                            [0., 1., 0.],
                            [1., 3., 1.],
                            [0., 1., 0.],
                            ]),
    "k3":  np.array([
                            [-1., -1., -1.],
                            [ 0.,  0.,  0.],
                            [ 1.,  1.,  1.],
                            ])
}


image = np.array([
                  [1., 0., 3.],
                  [6., 2., 7.],
                  [0., 8., 4.]
])
```

Rozwiązanie 1a — obliczenia ręczne

Stosuję krok 1 i dopełnienie jedną warstwą zer. Tak jak `Conv2D`, przesuwam filtr bez odwracania (ściśle: korelacja krzyżowa). Każdy wynik jest sumą dziewięciu iloczynów. Indeksy poniżej zaczynają się od 1.

Wejście po dopełnieniu:

```
[0, 0, 0, 0, 0]
[0, 1, 0, 3, 0]
[0, 6, 2, 7, 0]
[0, 0, 8, 4, 0]
[0, 0, 0, 0, 0]
```

Filtr k1

- `Y[1,1] = 0·(2) + 0·(0) + 0·(-2) + 0·(2) + 1·(0) + 0·(-2) + 0·(2) + 6·(0) + 2·(-2) = -4`
- `Y[1,2] = 0·(2) + 0·(0) + 0·(-2) + 1·(2) + 0·(0) + 3·(-2) + 6·(2) + 2·(0) + 7·(-2) = -6`
- `Y[1,3] = 0·(2) + 0·(0) + 0·(-2) + 0·(2) + 3·(0) + 0·(-2) + 2·(2) + 7·(0) + 0·(-2) = 4`
- `Y[2,1] = 0·(2) + 1·(0) + 0·(-2) + 0·(2) + 6·(0) + 2·(-2) + 0·(2) + 0·(0) + 8·(-2) = -20`
- `Y[2,2] = 1·(2) + 0·(0) + 3·(-2) + 6·(2) + 2·(0) + 7·(-2) + 0·(2) + 8·(0) + 4·(-2) = -14`
- `Y[2,3] = 0·(2) + 3·(0) + 0·(-2) + 2·(2) + 7·(0) + 0·(-2) + 8·(2) + 4·(0) + 0·(-2) = 20`
- `Y[3,1] = 0·(2) + 6·(0) + 2·(-2) + 0·(2) + 0·(0) + 8·(-2) + 0·(2) + 0·(0) + 0·(-2) = -20`
- `Y[3,2] = 6·(2) + 2·(0) + 7·(-2) + 0·(2) + 8·(0) + 4·(-2) + 0·(2) + 0·(0) + 0·(-2) = -10`
- `Y[3,3] = 2·(2) + 7·(0) + 0·(-2) + 8·(2) + 4·(0) + 0·(-2) + 0·(2) + 0·(0) + 0·(-2) = 20`

Macierz wynikowa:

```
[-4, -6, 4]
[-20, -14, 20]
[-20, -10, 20]
```

Filtr k2

- `Y[1,1] = 0·(0) + 0·(1) + 0·(0) + 0·(1) + 1·(3) + 0·(1) + 0·(0) + 6·(1) + 2·(0) = 9`
- `Y[1,2] = 0·(0) + 0·(1) + 0·(0) + 1·(1) + 0·(3) + 3·(1) + 6·(0) + 2·(1) + 7·(0) = 6`
- `Y[1,3] = 0·(0) + 0·(1) + 0·(0) + 0·(1) + 3·(3) + 0·(1) + 2·(0) + 7·(1) + 0·(0) = 16`
- `Y[2,1] = 0·(0) + 1·(1) + 0·(0) + 0·(1) + 6·(3) + 2·(1) + 0·(0) + 0·(1) + 8·(0) = 21`
- `Y[2,2] = 1·(0) + 0·(1) + 3·(0) + 6·(1) + 2·(3) + 7·(1) + 0·(0) + 8·(1) + 4·(0) = 27`
- `Y[2,3] = 0·(0) + 3·(1) + 0·(0) + 2·(1) + 7·(3) + 0·(1) + 8·(0) + 4·(1) + 0·(0) = 30`
- `Y[3,1] = 0·(0) + 6·(1) + 2·(0) + 0·(1) + 0·(3) + 8·(1) + 0·(0) + 0·(1) + 0·(0) = 14`
- `Y[3,2] = 6·(0) + 2·(1) + 7·(0) + 0·(1) + 8·(3) + 4·(1) + 0·(0) + 0·(1) + 0·(0) = 30`
- `Y[3,3] = 2·(0) + 7·(1) + 0·(0) + 8·(1) + 4·(3) + 0·(1) + 0·(0) + 0·(1) + 0·(0) = 27`

Macierz wynikowa:

```
[9, 6, 16]
[21, 27, 30]
[14, 30, 27]
```

Filtr k3

- `Y[1,1] = 0·(-1) + 0·(-1) + 0·(-1) + 0·(0) + 1·(0) + 0·(0) + 0·(1) + 6·(1) + 2·(1) = 8`
- `Y[1,2] = 0·(-1) + 0·(-1) + 0·(-1) + 1·(0) + 0·(0) + 3·(0) + 6·(1) + 2·(1) + 7·(1) = 15`

- `Y[1,3] = 0·(-1) + 0·(-1) + 0·(-1) + 0·(0) + 3·(0) + 0·(0) + 2·(1) + 7·(1) + 0·(1) = 9`
- `Y[2,1] = 0·(-1) + 1·(-1) + 0·(-1) + 0·(0) + 6·(0) + 2·(0) + 0·(1) + 0·(1) + 8·(1) = 7`
- `Y[2,2] = 1·(-1) + 0·(-1) + 3·(-1) + 6·(0) + 2·(0) + 7·(0) + 0·(1) + 8·(1) + 4·(1) = 8`
- `Y[2,3] = 0·(-1) + 3·(-1) + 0·(-1) + 2·(0) + 7·(0) + 0·(0) + 8·(1) + 4·(1) + 0·(1) = 9`
- `Y[3,1] = 0·(-1) + 6·(-1) + 2·(-1) + 0·(0) + 0·(0) + 8·(0) + 0·(1) + 0·(1) + 0·(1) = -8`
- `Y[3,2] = 6·(-1) + 2·(-1) + 7·(-1) + 0·(0) + 8·(0) + 4·(0) + 0·(1) + 0·(1) + 0·(1) = -15`
- `Y[3,3] = 2·(-1) + 7·(-1) + 0·(-1) + 8·(0) + 4·(0) + 0·(0) + 0·(1) + 0·(1) + 0·(1) = -9`

Macierz wynikowa:

```
[8, 15, 9]
[7, 8, 9]
[-8, -15, -9]
```

```
from IPython.display import display, Markdown
manual_results = {
    'k1': np.array([[-4, -6, 4], [-20, -14, 20], [-20, -10, 20]]),
    'k2': np.array([[9, 6, 16], [21, 27, 30], [14, 30, 27]]),
    'k3': np.array([[8, 15, 9], [7, 8, 9], [-8, -15, -9]]),
}

def conv2d_manual(array, kernel):
    """SAME, krok 1, nieparzyste jądro; bez gotowej funkcji splotu."""
    height, width = array.shape
    kh, kw = kernel.shape
    if kh % 2 != 1 or kw % 2 != 1:
        raise ValueError('W tej implementacji jądro musi mieć nieparzyste wymiary.')
    # Jawne zbudowanie dopełnienia zerami i skopiowanie wejścia.
    padded = np.zeros((height + kh - 1, width + kw - 1), dtype=float)
    for row in range(height):
        for col in range(width):
            padded[row + kh // 2, col + kw // 2] = array[row, col]
    output = np.zeros((height, width), dtype=float)
    # Każdy element wyniku obliczamy czterema zagnieżdżonymi pętlami.
    for row in range(height):
        for col in range(width):
            total = 0.0
            for kr in range(kh):
                for kc in range(kw):
                    total += padded[row + kr, col + kc] * kernel[kr, kc]
            output[row, col] = total
    return output

def correlate_same_numpy(array, kernel):
    padded_array = np.pad(array, 1, mode='constant')
    windows = np.lib.stride_tricks.sliding_window_view(padded_array, kernel.shape)
    return np.einsum('ijab,ab->ij', windows, kernel)

for name, kernel in kernels.items():
    result = conv2d_manual(image, kernel)
    np.testing.assert_allclose(result, manual_results[name])
    np.testing.assert_allclose(result, correlate_same_numpy(image, kernel))
    print(f'{name}: własne pętle = obliczenia ręczne = kontrola NumPy')
    print(result)
# Kontrola brzegów i orientacji filtra, również na niesymetrycznym wejściu.
np.testing.assert_array_equal(conv2d_manual(image, np.array([[1.]])), image)
np.testing.assert_array_equal(conv2d_manual(np.zeros((2, 4)), kernels['k1']), np.zeros((2, 4)))
```

```
k1: własne pętle = obliczenia ręczne = kontrola NumPy
[[ -4.  -6.   4.]
 [-20. -14.  20.]
 [-20. -10.  20.]]
k2: własne pętle = obliczenia ręczne = kontrola NumPy
[[ 9.  6. 16.]
 [21. 27. 30.]
 [14. 30. 27.]]
k3: własne pętle = obliczenia ręczne = kontrola NumPy
[[  8.  15.   9.]
 [  7.   8.   9.]
 [ -8. -15.  -9.]]
```

Zadanie 2. Wykorzystanie splotu różnych jąder na obrazach

Warstwy splotowe same uczą się wyodrębniania istotnych cech z danych - lokalnych wzorców. Wzorce te są rozpoznawane przez sieć niezależnie od ich położenia. Wagi poszczególnych neuronów reprezentowane są przez jądra splotowe (filtry), których rozmiar jest taki jak rozmiar pola recepcyjnego. Wybór wartości danego jądra determinuje to, jakie wzorce będą wykrywane.

2a. Wykorzystując funkcję *apply_conv2d*, która pozwala na obliczenie splotu jądra z obrazem wejściowym, przeprowadź eksperymenty wykorzystując zdefiniowane wcześniej jądra oraz obraz *lab1_1.jpg*. Sprawdź i wyświetl jakie cechy podkreślają dla obrazu w skali szarości poszczególne filtry - wyświetl obraz przed i po zastosowaniu splotu. Dodatkowo, stwórz jeden własny filtr 5x5 i również wyświetl efekt splotu dla tego jądra.

```python
import numpy as np
import tensorflow as tf

def apply_conv2d(input, kernel, strides=(1,1), padding='SAME'):

  #zmieniamy kształt danych na (batch, no_of_rows, no_of_columns, no_of_channels)
  input = input.astype(np.float64)
  input = input[np.newaxis, ...]

  #wymiary jądra
  k_width, k_height = kernel.shape
  #liczba wyjść
  no_of_filters = 1

  input_depth = 1

  #kształt jądra powinnien być: (kernel_width, kernel_height, input_depth, no_of_filters)
  k_shape=(k_width, k_height, input_depth, no_of_filters)

  kernel_c = kernel.reshape(k_shape)

  #tworzymy warstwę splotową 2D
  layer_2D_1 = tf.keras.layers.Conv2D(no_of_filters,
                                      kernel_size=(k_width, k_height),
                                      strides = strides,
                                      padding=padding,
                                      activation='linear',
                                      use_bias=False,
                                      kernel_initializer=tf.keras.initializers.Constant(kernel_c))

  return layer_2D_1(input).numpy()
```

```python
from tensorflow import keras

def read_image(filepath, h=None, w=None, color_mode='grayscale'):
    # Bez h i w zachowujemy oryginalne wymiary obrazu.
    if (h is None) != (w is None):
        raise ValueError('Podaj jednocześnie h i w albo pomiń oba wymiary.')
    target_size = None if h is None else (h, w)
```

```
    img = tf.keras.utils.load_img(filepath, target_size=target_size, color_mode=color_mode)
    return tf.keras.utils.img_to_array(img)
```

```
%matplotlib inline
import matplotlib.pyplot as plt
from PIL import Image

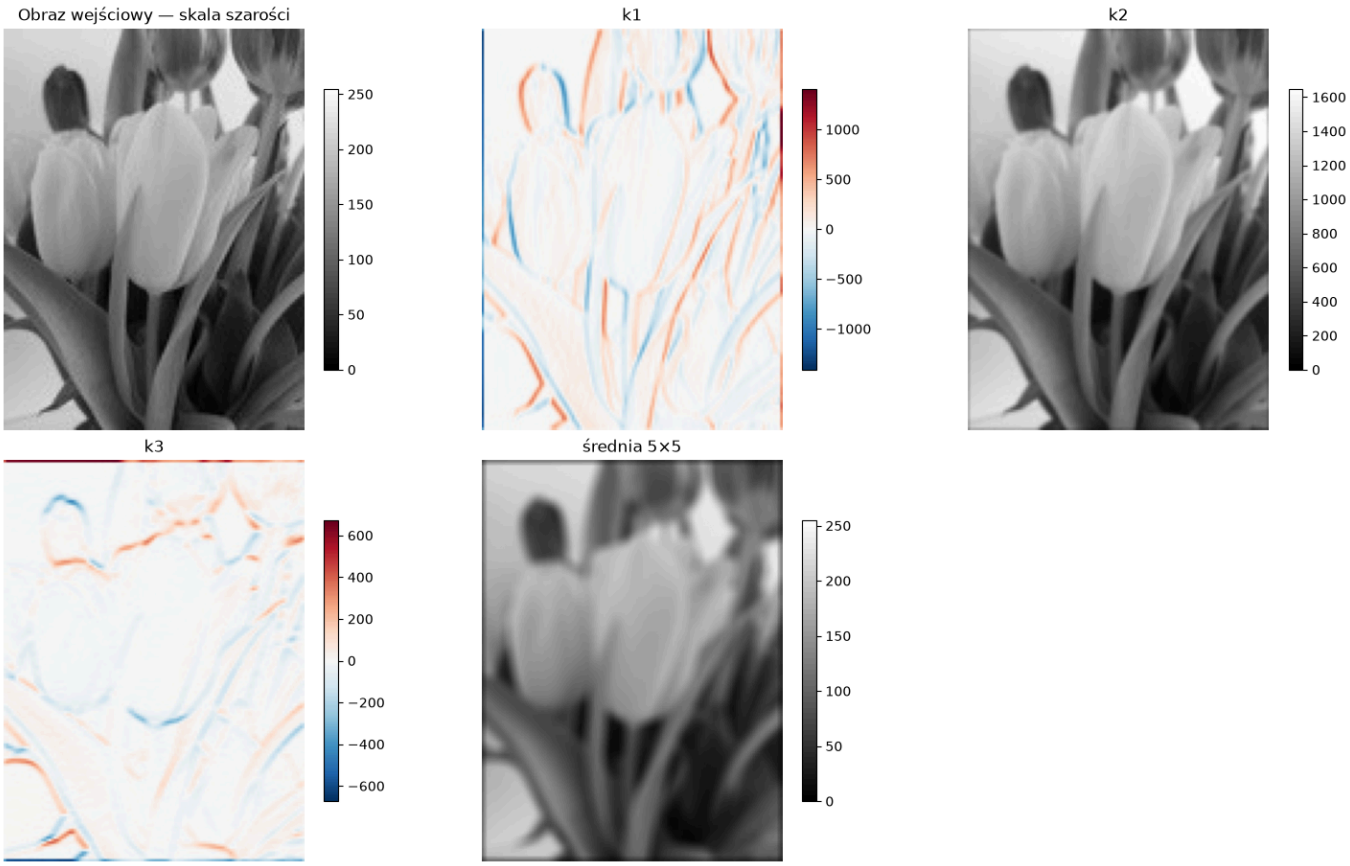
# Sprawdzenie zgodności z TensorFlow dla wszystkich trzech macierzy z 1a.
for name, kernel in kernels.items():
    actual = apply_conv2d(image[..., np.newaxis], kernel)[0, :, :, 0]
    np.testing.assert_allclose(actual, manual_results[name])
print('Wszystkie trzy wyniki z 1a są zgodne z NumPy i Conv2D.')

gray_image = read_image('lab1_1.jpg')
with Image.open('lab1_1.jpg') as original_image:
    assert gray_image.shape[:2] == (original_image.height, original_image.width)
print(f'Wymiary obrazu (wysokość, szerokość, kanały): {gray_image.shape}')

experiment_kernels = {**kernels, 'średnia 5×5': np.full((5, 5), 1 / 25)}
filtered_images = {
    name: apply_conv2d(gray_image, kernel)[0, :, :, 0]
    for name, kernel in experiment_kernels.items()
}
for result in filtered_images.values():
    assert result.shape == gray_image.shape[:2]
    assert np.isfinite(result).all()

fig, axes = plt.subplots(2, 3, figsize=(15, 9), layout='constrained')
axes = axes.ravel()
original_plot = axes[0].imshow(gray_image[:, :, 0], cmap='gray', vmin=0, vmax=255)
axes[0].set_title('Obraz wejściowy — skala szarości')
fig.colorbar(original_plot, ax=axes[0], shrink=0.7)
for ax, (name, result) in zip(axes[1:], filtered_images.items()):
    if name in ('k1', 'k3'):
        limit = max(float(np.max(np.abs(result))), 1.0)
        plot = ax.imshow(result, cmap='RdBu_r', vmin=-limit, vmax=limit)
    else:
        # Stały zakres dla średniej; pełny zakres dla wzmacniającego jasność k2.
        upper = 255 if name == 'średnia 5×5' else max(255, float(result.max()))
        plot = ax.imshow(result, cmap='gray', vmin=0, vmax=upper)
    ax.set_title(name)
    fig.colorbar(plot, ax=ax, shrink=0.7)
for ax in axes:
    ax.set_axis_off()
axes[-1].set_visible(False)
plt.show()
plt.close(fig)
```

```
Wszystkie trzy wyniki z 1a są zgodne z NumPy i Conv2D.
Wymiary obrazu (wysokość, szerokość, kanały): (160, 120, 1)
```



Wnioski z 2a

- **k1** porównuje jasność po lewej i prawej stronie: podkreśla pionowe krawędzie, m.in. boczne kontury płatków i łodyg tulipanów. Dodatnia odpowiedź oznacza jaśniejszą lewą stronę, ujemna — prawą. Jednolite wnętrza obszarów dają odpowiedź bliską zeru.
- **k3** porównuje dolne i górne sąsiedztwo: podkreśla poziome krawędzie, np. górne i dolne fragmenty obrysu kwiatów. Dodatnia odpowiedź oznacza jaśniejszy dół, ujemna — górę.
- **k2** sumuje sąsiadów i trzykrotnie waży środek. Wzmacnia jasność (suma wag wynosi 7), nie jest znormalizowanym filtrem uśredniającym. Na wykresie zachowano zakres wartości wykraczający poza 255, bez obcinania wyniku.
- **Własny filtr 5×5** ma 25 wag równych `1/25`, o sumie 1. Uśrednia otoczenie: wygładza detale i rozmywa granice. Przy krawędzi obrazu zerowe dopełnienie obniża średnią i może tworzyć ciemniejszy brzeg.

Mapy k1 i k3 mają symetryczną skalę wokół zera: oba znaki odpowiedzi są widoczne. Dopełnienie zerami może też tworzyć odpowiedź na zewnętrznym brzegu obrazu, nawet jeśli nie jest to krawędź obiektu.

2b. Funkcja *apply_con2d* pozwala również na zdefiniowanie kroku oraz tego, czy wykorzystujemy dopełnienie zerami dla wartości położonych na krawędziach. Sprawdź, jak zmieniają się wymiary wyjścia po zastosowaniu splotu obrazu wejściowego z jądrem, stosując (lub nie) dopełnianie zerami oraz zmianę wartości kroku. Zapisz swoje wnioski.

```
height, width = gray_image.shape[:2]
kernel_height, kernel_width = kernels['k1'].shape
dimension_rows = []
for padding in ('SAME', 'VALID'):
    for stride in (1, 2, 3):
        output = apply_conv2d(gray_image, kernels['k1'],
                              strides=(stride, stride), padding=padding)
        if padding == 'SAME':
            expected = ((height + stride - 1) // stride,
                        (width + stride - 1) // stride)
        else:
            expected = ((height - kernel_height) // stride + 1,
                        (width - kernel_width) // stride + 1)
        actual = output.shape[1:3]
```

```
        assert actual == expected, (padding, stride, actual, expected)
        assert output.shape[0] == output.shape[3] == 1
        dimension_rows.append((padding, stride, expected, actual))

table = ['| Padding | Krok (H, W) | Obliczone (H, W) | Rzeczywiste (H, W) |',
         '|---|---|---|---|']
for padding, stride, expected, actual in dimension_rows:
    table.append(f'| {padding} | ({stride}, {stride}) | {expected} | {actual} |')
display(Markdown('\n'.join(table)))
print('Wszystkie 6 wariantów ma wymiary zgodne ze wzorami.')
```

Padding	Krok (H, W)	Obliczone (H, W)	Rzeczywiste (H, W)
SAME	(1, 1)	(160, 120)	(160, 120)
SAME	(2, 2)	(80, 60)	(80, 60)
SAME	(3, 3)	(54, 40)	(54, 40)
VALID	(1, 1)	(158, 118)	(158, 118)
VALID	(2, 2)	(79, 59)	(79, 59)
VALID	(3, 3)	(53, 40)	(53, 40)

```
Wszystkie 6 wariantów ma wymiary zgodne ze wzorami.
```

Wzory i wnioski z 2b

Dla każdego wymiaru osobno (`N` — wysokość lub szerokość wejścia, `K` — odpowiedni wymiar filtra, `S` — krok), przy dylatacji 1:

- **SAME:** $N_{out} = \lceil N/S \rceil$.
- **VALID:** $N_{out} = \lfloor (N - K)/S \rfloor + 1$.

Zatem `H_out` liczymy z `H, K_H, S_H` , a `W_out` z `W, K_W, S_W` . W eksperymencie `K_H = K_W = 3` .

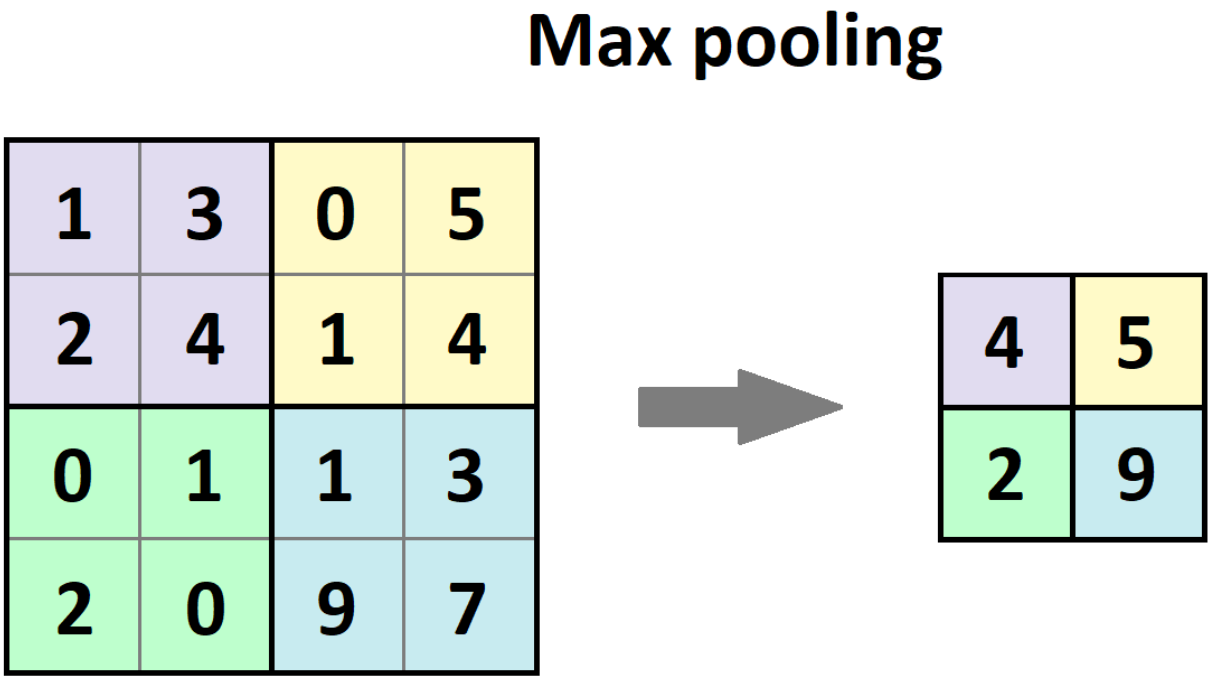
`SAME` zachowuje wymiary wejścia tylko przy kroku 1. Przy większym kroku również zmniejsza wyjście. TensorFlow dobiera liczbę zer tak, aby uzyskać powyższy wymiar; dla większego kroku dopełnienie może być asymetryczne lub nie być potrzebne.

`VALID` nie dodaje zer i korzysta wyłącznie z pełnych okien mieszczących się w obrazie. Dla kroku 1 filtr 3×3 zmniejsza oba wymiary o 2. Większy krok oznacza rzadsze próbkowanie, mniejsze wyjście i możliwość pominięcia części szczegółów. Wyniki tabeli potwierdzają wzory dla wszystkich sześciu konfiguracji.

Zadanie 3. Obliczanie rozmiaru wyjściowego modelu

Warstwy *pooling* są kolejny elementem budującym sieć splotową. Zmniejszają wymiar przestrzenny, co pomaga ograniczyć ryzyko przetrenowania (zmniejsza się liczba parametrów), działają na każdym z kanałów (każdej mapie cech), więc głębokość wyjściowa jest taka sama (zmienia się tylko wymiar przestrzenny). Warstwa ta nie ma wag.

Na poniższym rysunku przedstawiona została operacja *max pooling* z krokiem 2 oraz wymiarem jądra 2, czyli wybór maksymalnej wartości z pola recepcyjnego - pozostałe wartości pozostają odrzucone. W takim podejściu odrzucamy wiele wartości, ale pozostawiamy tylko ekstremalne, najbardziej znaczące cechy.



Stosując warstwę *average pooling* uzyskamy taki sam efekt redukcji wymiarów, jednak zamiast wartości maksymalnej z pola recepcyjnego, obliczmy wartość średnią w tym polu. Przy tym podejściu możemy uzyskać zbyt uogólnione wyjście.

3a. Analizując poniżej przedstawiony przykład sekwencji operacji na podanym wejściu, zapisz jaki jest kształt wyjściowy dla każdego z poszczególnych kroków.

input = np.ones((24, 24, 1))

1. Splot z 16 filtrami o rozmiarze 3x3, krok (1,1), padding SAME
2. Pooling - jądro 5x5, krok (3,3), padding VALID
3. Splot z 32 filtrami o rozmiarze 3x3, krok (1,1), padding SAME
4. Pooling - jądro 3x3, krok (2,2), padding VALID

```
expected_shapes = [(24, 24, 16), (7, 7, 16), (7, 7, 32), (3, 3, 32)]
display(Markdown('\n| Krok | Obliczenie wymiaru przestrzennego | Kształt H×W×C |\n|---|---|---|\n| Wejście | – | 24×24×1 |\n| Splot 3×3, SAME, 16 filtrów | ceil(24/1) = 24 | 24×24×16 |\n| Pooling 5×5, krok 3, VALID | floor((24−5)/3)+1 = 7 | 7×7×16 |\n| Splot 3×3, SAME, 32 filtry | ceil(7/1) = 7 | 7×7×32 |\n| Pooling 3×3, krok 2, VALID | floor((7−3)/2)+1 = 3 | 3×3×32 |\n\nPooling zachowuje liczbę kanałów. Splot ustala ją na liczbę filtrów.\n'))
layers_3a = [
    tf.keras.layers.Conv2D(16, 3, padding='same'),
    tf.keras.layers.MaxPooling2D(5, strides=3, padding='valid'),
    tf.keras.layers.Conv2D(32, 3, padding='same'),
    tf.keras.layers.MaxPooling2D(3, strides=2, padding='valid'),
]
tensor = tf.ones((1, 24, 24, 1))
for index, (layer, expected) in enumerate(zip(layers_3a, expected_shapes), 1):
    tensor = layer(tensor)
    assert tuple(tensor.shape[1:]) == expected
    print(f'Krok {index}: {tuple(tensor.shape)} (z wymiarem batch) – zgodne')
```

Krok	Obliczenie wymiaru przestrzennego	Kształt H×W×C
Wejście	—	24×24×1
Splot 3×3, SAME, 16 filtrów	ceil(24/1) = 24	24×24×16
Pooling 5×5, krok 3, VALID	floor((24−5)/3)+1 = 7	7×7×16
Splot 3×3, SAME, 32 filtry	ceil(7/1) = 7	7×7×32
Pooling 3×3, krok 2, VALID	floor((7−3)/2)+1 = 3	3×3×32

Pooling zachowuje liczbę kanałów. Splot ustala ją na liczbę filtrów.

```
Krok 1: (1, 24, 24, 16) (z wymiarem batch) — zgodne
Krok 2: (1, 7, 7, 16) (z wymiarem batch) — zgodne
Krok 3: (1, 7, 7, 32) (z wymiarem batch) — zgodne
Krok 4: (1, 3, 3, 32) (z wymiarem batch) — zgodne
```

Zadanie 4. Implementacja i trenowanie prostej sieci splotowej

Początkowym etapem jest przygotowanie zbioru danych. W tym zadaniu wykorzystany zostanie zbiór danych *CIFAR10*.

Założenia porównania modeli

Korzystam z całego CIFAR-10: 50 000 obrazów do uczenia i walidacji oraz 10 000 niezależnych obrazów testowych. Z pierwszego zbioru wydzielam stratyfikowaną walidację 5 000 obrazów (po 500 na klasę), pozostawiając 45 000 do aktualizacji wag. Wszystkie modele korzystają z identycznego podziału. Każdy trenuje przez 15 epok, batch size 64, optymalizator Adam z learning rate 0.001 (wartość domyślna optymalizatora z przykładu). Nieużywane `learning_rate = 0.01` z szablonu zastępuję jawną konfiguracją. Wagi najlepszej epoki wybieram według `val_accuracy`, nigdy według wyniku testowego. Ustalone ziarno to 42.

4a. Wczytaj dane ze zbioru *CIFAR10* i podziel je na dane treningowe i testowe. Przygotuj listę zawierającą etykiety poszczególnych klas. Wyświetl przykładowy obraz należący do jednej z klas.

> Dokumentacja - zbiór CIFAR10

```
SEED = 42
tf.keras.utils.set_random_seed(SEED)
tf.config.experimental.enable_op_determinism()
# Ten sam plik i hash co w instrukcji; szybszy oficjalny mirror TensorFlow.
_ = keras.utils.get_file(
    fname='cifar-10-batches-py-target',
    origin='https://storage.googleapis.com/tensorflow/tf-keras-datasets/cifar-10-python.tar.gz',
    extract=True,
    file_hash='6d958be074577803d12ecdefd02955f39262c83c16fe9348329d7fe0b5c001ce',
)
(x_train_raw, y_train_raw), (x_test_raw, y_test_raw) = keras.datasets.cifar10.load_data()
class_names = ['samolot', 'samochód', 'ptak', 'kot', 'jeleń',
               'pies', 'żaba', 'koń', 'statek', 'ciężarówka']
assert x_train_raw.shape == (50000, 32, 32, 3)
assert x_test_raw.shape == (10000, 32, 32, 3)
assert set(y_train_raw.ravel()) == set(range(10))
print('Trening:', x_train_raw.shape, 'Test:', x_test_raw.shape)
print('Klasy:', dict(enumerate(class_names)))
```

```
fig, ax = plt.subplots(figsize=(3, 3))
ax.imshow(x_train_raw[0])
ax.set_title(f'Przykład: {class_names[int(y_train_raw[0, 0])]}')
ax.axis('off')
plt.show()
plt.close(fig)
```

```
Trening: (50000, 32, 32, 3) Test: (10000, 32, 32, 3)
Klasy: {0: 'samolot', 1: 'samochód', 2: 'ptak', 3: 'kot', 4: 'jeleń', 5: 'pies', 6: 'żaba', 7: 'koń', 8: 'statek',
9: 'ciężarówka'}
```



Kolejnym etapem jest sprawdzenie i przygotowanie danych do treningu sieci.

```
# Wydzielamy po 500 przykładów każdej klasy na walidację, tylko ze zbioru treningowego.
rng = np.random.default_rng(SEED)
train_indices, val_indices = [], []
for label in range(10):
    indices = np.flatnonzero(y_train_raw.ravel() == label)
    rng.shuffle(indices)
    val_indices.extend(indices[:500])
    train_indices.extend(indices[500:])
train_indices = rng.permutation(train_indices)
val_indices = np.array(val_indices)
assert len(set(train_indices) & set(val_indices)) == 0
assert len(train_indices) + len(val_indices) == 50000
x_train = x_train_raw[train_indices].astype('float32') / 255.0
x_val = x_train_raw[val_indices].astype('float32') / 255.0
x_test = x_test_raw.astype('float32') / 255.0
y_train = keras.utils.to_categorical(y_train_raw[train_indices], 10)
y_val = keras.utils.to_categorical(y_train_raw[val_indices], 10)
y_test = keras.utils.to_categorical(y_test_raw, 10)
for x, y in [(x_train, y_train), (x_val, y_val), (x_test, y_test)]:
    assert x.dtype == np.float32 and 0 <= x.min() <= x.max() <= 1
    assert y.shape == (len(x), 10)
    np.testing.assert_allclose(y.sum(axis=1), 1)
print('Trening:', x_train.shape, 'Walidacja:', x_val.shape, 'Test:', x_test.shape)
```

```
Trening: (45000, 32, 32, 3) Walidacja: (5000, 32, 32, 3) Test: (10000, 32, 32, 3)
```

W kolejnym kroku zdefiniujmy podstawowe parametry treningu.

```
batch_size = 64
epochs = 15
no_of_classes = 10
learning_rate = 0.001  # domyślne Adam z przykładu; jawnie używane w każdym modelu
ARTIFACTS = Path('artifacts')
ARTIFACTS.mkdir(exist_ok=True)
```

```
def make_dataset(x, y, training=False):
    ds = tf.data.Dataset.from_tensor_slices((x, y))
    if training:
        ds = ds.shuffle(len(x), seed=SEED, reshuffle_each_iteration=True)
    options = tf.data.Options()
    options.threading.private_threadpool_size = 2
    options.threading.max_intra_op_parallelism = 1
    return ds.batch(batch_size).with_options(options).prefetch(1)

def compile_model(model):
    model.compile(optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
                  loss='categorical_crossentropy', metrics=['accuracy'])

def train_model(model, name):
    # Najlepsza epoka wybierana tylko wg walidacji. Każdy model trenuje pełne 15 epok.
    checkpoint = ARTIFACTS / f'{name}.weights.h5'
    hist = model.fit(make_dataset(x_train, y_train, True),
                     validation_data=make_dataset(x_val, y_val), epochs=epochs,
                     callbacks=[keras.callbacks.ModelCheckpoint(
                         str(checkpoint), monitor='val_accuracy', mode='max',
                         save_best_only=True, save_weights_only=True),
                                keras.callbacks.CSVLogger(str(ARTIFACTS / f'{name}.csv'))], verbose=2)
    model.load_weights(checkpoint)
    assert len(hist.history['accuracy']) == epochs
    assert all(np.isfinite(values).all() for values in hist.history.values())
    return hist

def plot_history(hist, title):
    fig, axes = plt.subplots(1, 2, figsize=(11, 3.5), layout='constrained')
    for ax, metric, ylabel in zip(axes, ['accuracy', 'loss'], ['Dokładność', 'Strata']):
        ax.plot(range(1, epochs+1), hist.history[metric], label='trening')
        ax.plot(range(1, epochs+1), hist.history['val_'+metric], label='walidacja')
        ax.set(xlabel='Epoka', ylabel=ylabel)
        ax.legend()
        ax.grid(alpha=0.3)
    fig.suptitle(title)
    plt.show()
    plt.close(fig)
```

Po przygotowaniu danych, wykorzystując API Keras, zdefiniujmy prosty model oraz wyświetlmy architekturę stworzonej sieci.

```
tf.keras.utils.set_random_seed(SEED)
keras_model = keras.Sequential([
    keras.Input(shape=(32, 32, 3)),
    keras.layers.Conv2D(2, 4, padding='same', activation='relu', name='base_conv'),
    keras.layers.Flatten(),
    keras.layers.Dense(50, activation='relu'),
    keras.layers.Dense(no_of_classes, activation='softmax'),
], name='model_bazowy')
keras_model.summary()
```

Model: "model_bazowy"

Layer (type)	Output Shape	Param #
base_conv (Conv2D)	(None, 32, 32, 2)	98
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 50)	102,450
dense_1 (Dense)	(None, 10)	510

```
Total params: 103,058 (402.57 KB)
```

```
Trainable params: 103,058 (402.57 KB)
```

```
Non-trainable params: 0 (0.00 B)
```

Dla utworzonego powyżej modelu, kształt wyjściowy z pierwszej warstwy konwolucyjnej jest taki sam jak kształt wejściowy (zastosowane zostało dopełnienie zerami), a głębokość wyjściowa jest równa liczbie ustawionych filtrów - 2.

W kolejnym etapie ustawmy parametry treningu i rozpocznijmy uczenie modelu na danych treningowych.

```
compile_model(keras_model)
history = train_model(keras_model, 'baseline')
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
```

```
Epoch 1/15
```

```
C:\Users\igor\source\multimodal-ai\deep-learning\lab1\.venv\Lib\site-packages\keras\src\trainers\epoch_iterator.py:74: UserWarning: `shuffle=True` was passed, but will be ignored since the data `x` was provided as a tf.data.Dataset. The Dataset is expected to already be shuffled (via `.shuffle(buffer_size)`).
  self.data_adapter = data_adapters.get_data_adapter(
```

```
704/704 - 4s - 5ms/step - accuracy: 0.3339 - loss: 1.8708 - val_accuracy: 0.3954 - val_loss: 1.7119
```

```
Epoch 2/15
```

```
704/704 - 2s - 2ms/step - accuracy: 0.4204 - loss: 1.6459 - val_accuracy: 0.4158 - val_loss: 1.6555
```

```
Epoch 3/15
```

```
704/704 - 2s - 2ms/step - accuracy: 0.4495 - loss: 1.5573 - val_accuracy: 0.4202 - val_loss: 1.6397
```

```
Epoch 4/15
```

```
704/704 - 2s - 2ms/step - accuracy: 0.4720 - loss: 1.5006 - val_accuracy: 0.4482 - val_loss: 1.5613
```

```
Epoch 5/15
```

```
704/704 - 2s - 2ms/step - accuracy: 0.4882 - loss: 1.4539 - val_accuracy: 0.4590 - val_loss: 1.5233
```

```
Epoch 6/15
```

```
704/704 - 2s - 3ms/step - accuracy: 0.5054 - loss: 1.4079 - val_accuracy: 0.4612 - val_loss: 1.5232

Epoch 7/15

704/704 - 2s - 2ms/step - accuracy: 0.5168 - loss: 1.3701 - val_accuracy: 0.4516 - val_loss: 1.5356

Epoch 8/15

704/704 - 2s - 2ms/step - accuracy: 0.5311 - loss: 1.3366 - val_accuracy: 0.4494 - val_loss: 1.5782

Epoch 9/15

704/704 - 2s - 2ms/step - accuracy: 0.5391 - loss: 1.3080 - val_accuracy: 0.4558 - val_loss: 1.5374

Epoch 10/15

704/704 - 2s - 3ms/step - accuracy: 0.5523 - loss: 1.2743 - val_accuracy: 0.4558 - val_loss: 1.5767

Epoch 11/15

704/704 - 2s - 2ms/step - accuracy: 0.5626 - loss: 1.2487 - val_accuracy: 0.4554 - val_loss: 1.5559

Epoch 12/15

704/704 - 2s - 3ms/step - accuracy: 0.5709 - loss: 1.2217 - val_accuracy: 0.4616 - val_loss: 1.5621

Epoch 13/15

704/704 - 2s - 2ms/step - accuracy: 0.5780 - loss: 1.1967 - val_accuracy: 0.4610 - val_loss: 1.5525

Epoch 14/15

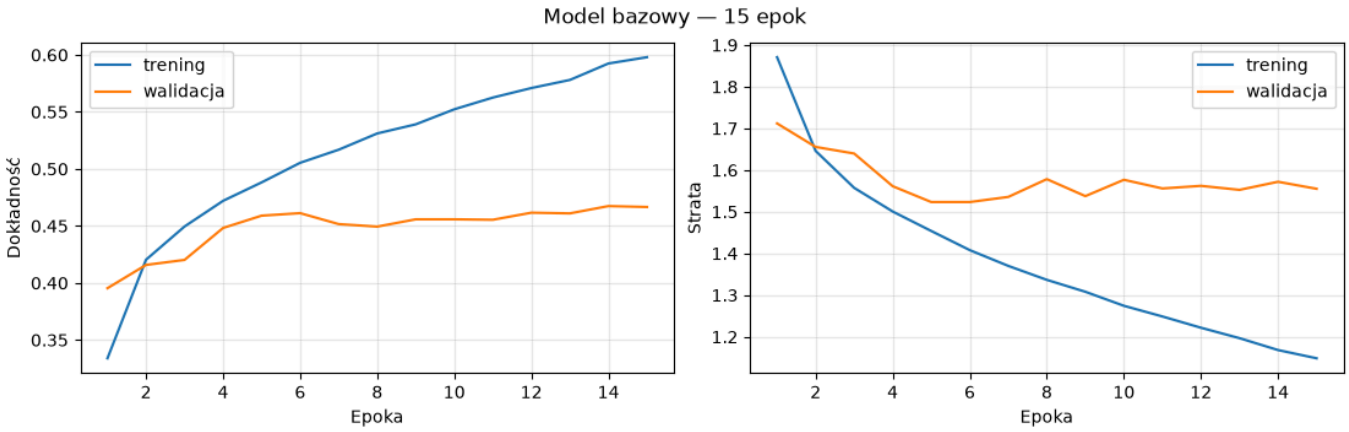
704/704 - 2s - 2ms/step - accuracy: 0.5924 - loss: 1.1681 - val_accuracy: 0.4674 - val_loss: 1.5719

Epoch 15/15

704/704 - 2s - 3ms/step - accuracy: 0.5980 - loss: 1.1484 - val_accuracy: 0.4666 - val_loss: 1.5551
```

Uzyskany wynik dokładności nie jest zadowalający, jednak stworzony model jest bardzo prosty i trenowany był przez 15 epok. Wyświetlmy wykres z historii, aby zobaczyć jak zmieniała się dokładność modelu.

```
plot_history(history, 'Model bazowy – 15 epok')
```



Sprawdźmy, jakie wartości metryk ewaluacyjnych osiąga wytrenowany model dla danych testowych.

```
loss, accuracy = keras_model.evaluate(make_dataset(x_test, y_test), verbose=0)
print(f'Model bazowy — test loss: {loss:.4f}, accuracy: {accuracy:.2%}')
```

```
Model bazowy — test loss: 1.5954, accuracy: 45.57%
```

4b. Stwórz nowy model, poprzez rozszerzenie modelu *keras_model* o dodatkową warstwę konwolucyjną (liczba filtrów: 32, kernel_size=3, dopełnianie zerami) oraz dwie warstwy max pooling (2,2). W pierwszej warstwie splotowej zwiększ liczbę filtrów do 8. Przeprowadź trening modelu (z parametrami takim jak dla pierwszej architektury) i porównaj uzyskane wyniki z prostym modelem, zawierającym tylko jedną warstwę splotową - zamieść komentarz.

```
tf.keras.utils.set_random_seed(SEED)
keras_model_2 = keras.Sequential([
    keras.Input(shape=(32, 32, 3)),
    keras.layers.Conv2D(8, 4, padding='same', activation='relu', name='conv_first'),
    keras.layers.MaxPooling2D(2),
    keras.layers.Conv2D(32, 3, padding='same', activation='relu', name='conv_second'),
    keras.layers.MaxPooling2D(2),
    keras.layers.Flatten(),
    keras.layers.Dense(50, activation='relu'),
    keras.layers.Dense(no_of_classes, activation='softmax'),
], name='model_4b')
keras_model_2.summary()
compile_model(keras_model_2)
history_2 = train_model(keras_model_2, 'model_4b')
loss_2, accuracy_2 = keras_model_2.evaluate(make_dataset(x_test, y_test), verbose=0)
plot_history(history_2, 'Model 4b — dodatkowy splot i pooling')
delta = 100 * (accuracy_2 - accuracy)
display(Markdown(f'\n**Porównanie 4b:** model bazowy: **{accuracy:.2%}**, loss **{loss:.4f}**;\nmodel 4b: **
{accuracy_2:.2%}**, loss **{loss_2:.4f}** na tych samych 10 000 obrazach testowych.\nZmiana dokładności: **
{delta:+.2f} punktów procentowych**.\n\nModel 4b ma więcej map cech i dwa poziomy ekstrakcji wzorców. Pooling
zmniejsza\nmapy 32×32 → 16×16 → 8×8, ograniczając rozmiar wejścia warstwy gęstej.\nPorównanie dotyczy jednego
uruchomienia z tym samym podziałem, Adam, krokiem\nuczenia 0.001, batch size 64 i 15 epokami. W obu przypadkach
wybrano wagi\nnajlepszej epoki według walidacji. Różnica trening–walidacja na wykresie\npokazuje ewentualne
przeuczenie; wynik pojedynczej próby nie jest dowodem\nstatystycznym przewagi architektury.\n'))
```

```
Model: "model_4b"
```

Layer (type)	Output Shape	Param #
conv_first (Conv2D)	(None, 32, 32, 8)	392
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 8)	0
conv_second (Conv2D)	(None, 16, 16, 32)	2,336
max_pooling2d_3 (MaxPooling2D)	(None, 8, 8, 32)	0
flatten_1 (Flatten)	(None, 2048)	0
dense_2 (Dense)	(None, 50)	102,450
dense_3 (Dense)	(None, 10)	510

Total params: 105,688 (412.84 KB)

Trainable params: 105,688 (412.84 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/15

704/704 - 5s - 7ms/step - accuracy: 0.4123 - loss: 1.6316 - val_accuracy: 0.4842 - val_loss: 1.4305

Epoch 2/15

704/704 - 3s - 5ms/step - accuracy: 0.5326 - loss: 1.3216 - val_accuracy: 0.5542 - val_loss: 1.2606

Epoch 3/15

704/704 - 3s - 5ms/step - accuracy: 0.5773 - loss: 1.2043 - val_accuracy: 0.5792 - val_loss: 1.2191

Epoch 4/15

704/704 - 4s - 6ms/step - accuracy: 0.6073 - loss: 1.1205 - val_accuracy: 0.5890 - val_loss: 1.1829

Epoch 5/15

704/704 - 3s - 5ms/step - accuracy: 0.6314 - loss: 1.0539 - val_accuracy: 0.6224 - val_loss: 1.0699

Epoch 6/15

```
704/704 - 3s - 4ms/step - accuracy: 0.6494 - loss: 1.0024 - val_accuracy: 0.6404 - val_loss: 1.0323

Epoch 7/15

704/704 - 3s - 4ms/step - accuracy: 0.6655 - loss: 0.9610 - val_accuracy: 0.6430 - val_loss: 1.0313

Epoch 8/15

704/704 - 3s - 5ms/step - accuracy: 0.6763 - loss: 0.9302 - val_accuracy: 0.6638 - val_loss: 0.9859

Epoch 9/15

704/704 - 3s - 5ms/step - accuracy: 0.6882 - loss: 0.9030 - val_accuracy: 0.6578 - val_loss: 1.0047

Epoch 10/15

704/704 - 3s - 5ms/step - accuracy: 0.6968 - loss: 0.8751 - val_accuracy: 0.6700 - val_loss: 0.9717

Epoch 11/15

704/704 - 3s - 4ms/step - accuracy: 0.7024 - loss: 0.8526 - val_accuracy: 0.6612 - val_loss: 0.9926

Epoch 12/15

704/704 - 3s - 4ms/step - accuracy: 0.7098 - loss: 0.8315 - val_accuracy: 0.6474 - val_loss: 1.0199

Epoch 13/15

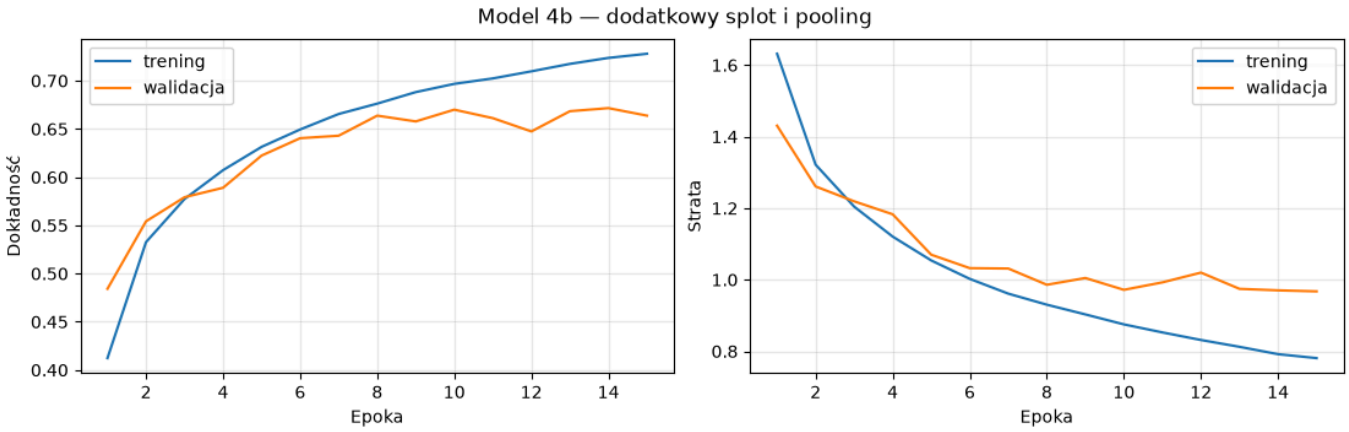
704/704 - 3s - 4ms/step - accuracy: 0.7175 - loss: 0.8124 - val_accuracy: 0.6684 - val_loss: 0.9744

Epoch 14/15

704/704 - 3s - 5ms/step - accuracy: 0.7237 - loss: 0.7919 - val_accuracy: 0.6716 - val_loss: 0.9703

Epoch 15/15

704/704 - 3s - 5ms/step - accuracy: 0.7280 - loss: 0.7808 - val_accuracy: 0.6638 - val_loss: 0.9674
```



Porównanie 4b: model bazowy: **45.57%**, loss **1.5954**;
model 4b: **66.02%**, loss **0.9797** na tych samych 10 000 obrazach testowych.
Zmiana dokładności: **+20.45 punktów procentowych**.

Model 4b ma więcej map cech i dwa poziomy ekstrakcji wzorców. Pooling zmniejsza mapy 32×32 → 16×16 → 8×8, ograniczając rozmiar wejścia warstwy gęstej. Porównanie dotyczy jednego uruchomienia z tym samym podziałem, Adam, krokiem uczenia 0.001, batch size 64 i 15 epokami. W obu przypadkach wybrano wagi najlepszej epoki według walidacji. Różnica trening–walidacja na wykresie pokazuje ewentualne przeuczenie; wynik pojedynczej próby nie jest dowodem statystycznym przewagi architektury.

Mając wytrenowany model możemy sprawdzić, jakich wzorców zdołał się nauczyć, szczególnie dla warstw splotowych.

Przyglądając się architekturze modelu, spróbujmy wyświetlić wagi oraz dane wyjściowe dla pierwszej warstwy konwolucyjnej modelu drugiego. Stwórzmy model, który po podaniu na jego wejście obrazu, zwróci wartości aktywacji dla modelu oryginalnego. Model będzie przyjmował jeden obiekt wejściowy i generował jeden obiekt wyjściowy dla wybranej warstwy.

```
first_conv = keras_model_2.get_layer('conv_first')
layer_output = first_conv.output
layer_weights = first_conv.get_weights()
activation_model = keras.Model(inputs=keras_model_2.inputs[0], outputs=layer_output)
assert layer_weights[0].shape == (4, 4, 3, 8)
```

Sprawdźmy, jak będzie wyglądało wyjście dla przykładowego obrazu ze zbioru testowego.

```
test_img = x_test[46:47]
activations = activation_model(test_img, training=False).numpy()
assert activations.shape == (1, 32, 32, 8)
print('Kształt aktywacji:', activations.shape)
```

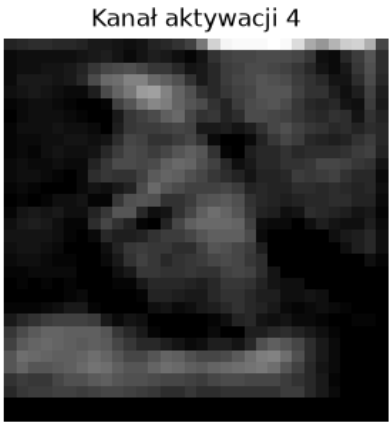
```
Kształt aktywacji: (1, 32, 32, 8)
```

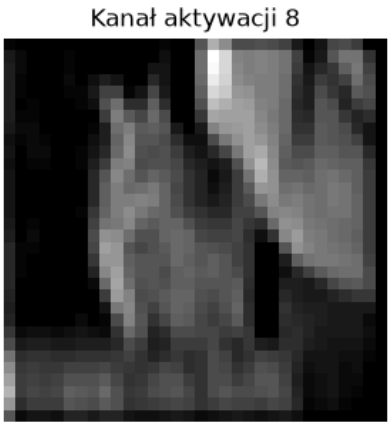
Na wyjściu otrzymujemy mapę cech o wymiarach 32x32, składającą się z 8 kanałów. Wyświetlmy w formie graficznej 4 i 8 kanał aktywacji tej warstwy oraz obraz wejściowy.

```
fig, axs = plt.subplots(1, 3, figsize=(10, 3), layout='constrained')
axs[0].imshow(test_img[0])
axs[0].set_title('Wejście – ' + class_names[int(y_test_raw[46, 0])])
for ax, channel in zip(axs[1:], [3, 7]):
    ax.imshow(activations[0, :, :, channel], cmap='gray')
    ax.set_title(f'Kanał aktywacji {channel+1}')
```

```
for ax in axs:
    ax.axis('off')
plt.show()
plt.close(fig)
```



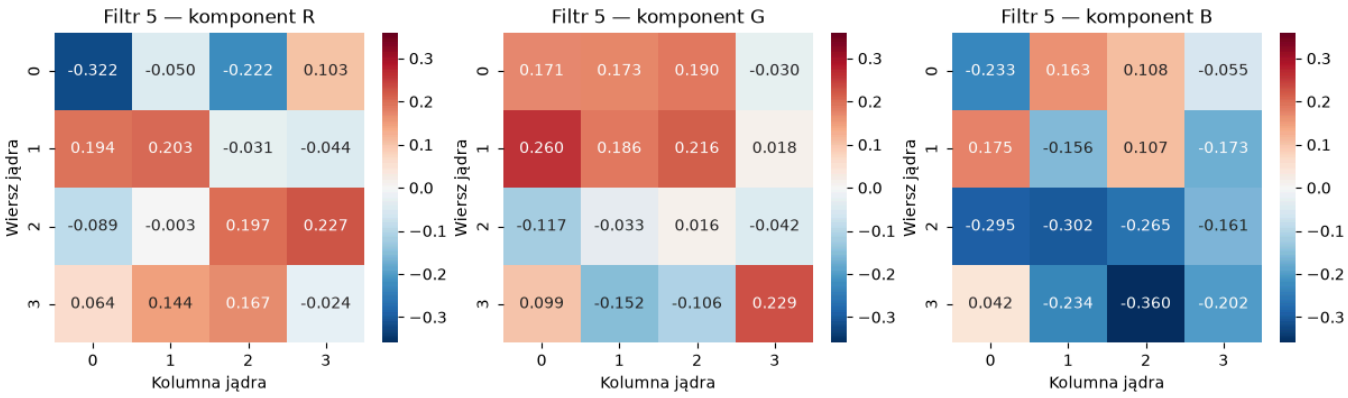




4c. Wyświetl wagi dla pierwszej warstwy splotowej, dla każdego z kanałów (komponentów) filtra 5 z wykorzystaniem map ciepła.

> Dokumentacja - wyświetlanie danych w postaci mapy ciepła

```
import seaborn as sns
weights, biases = first_conv.get_weights()
filter_index = 4  # piąty filtr, indeksowanie od zera
filter_weights = weights[:, :, :, filter_index]
assert filter_weights.shape == (4, 4, 3)
limit = float(np.max(np.abs(filter_weights)))
fig, axes = plt.subplots(1, 3, figsize=(12, 3.5), layout='constrained')
for channel, (ax, label) in enumerate(zip(axes, ['R', 'G', 'B'])):
    sns.heatmap(filter_weights[:, :, channel], ax=ax, annot=True, fmt='.3f',
                cmap='RdBu_r', center=0, vmin=-limit, vmax=limit, square=True)
    ax.set_title(f'Filtr 5 — komponent {label}')
    ax.set(xlabel='Kolumna jądra', ylabel='Wiersz jądra')
plt.show()
plt.close(fig)
print(f'Bias filtra 5: {biases[filter_index]:.5f}')
display(Markdown('Każdy filtr obejmuje wszystkie 3 kanały RGB. Pokazano '
                 'trzy macierze 4×4 piątego filtra, a nie piąty kanał aktywacji. '
                 'Wspólna skala pozwala porównać znaki i siłę wag.'))
```



Bias filtra 5: 0.12037

Każdy filtr obejmuje wszystkie 3 kanały RGB. Pokazano trzy macierze 4×4 piątego filtra, a nie piąty kanał aktywacji. Wspólna skala pozwala porównać znaki i siłę wag.

4d. Zmieniając parametry w warstwach splotowych oraz pooling (w modelu utworzonym w punkcie 4b) lub dodając dodatkowe warstwy splotowe/pooling, spróbuj utworzyć model, który uzyska większą dokładność na danych testowych.

```
tf.keras.utils.set_random_seed(SEED)
keras_model_3 = keras.Sequential([
    keras.Input(shape=(32, 32, 3)),
    keras.layers.Conv2D(32, 3, padding='same', activation='relu'),
    keras.layers.MaxPooling2D(2),
    keras.layers.Conv2D(64, 3, padding='same', activation='relu'),
    keras.layers.MaxPooling2D(2),
    keras.layers.Conv2D(128, 3, padding='same', activation='relu'),
    keras.layers.MaxPooling2D(2),
    keras.layers.Flatten(),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dropout(0.4, seed=SEED),
    keras.layers.Dense(no_of_classes, activation='softmax'),
], name='model_4d')
keras_model_3.summary()
compile_model(keras_model_3)
history_3 = train_model(keras_model_3, 'model_4d')
plot_history(history_3, 'Model 4d — trzy bloki splot/pooling i dropout')
# Architektura ustalona przed ewaluacją testową; nie wybieramy jej na podstawie testu.
loss_3, accuracy_3 = keras_model_3.evaluate(make_dataset(x_test, y_test), verbose=0)
rows = []
for label, model, hist, test_loss, test_acc in [
    ('Bazowy', keras_model, history, loss, accuracy),
    ('4b', keras_model_2, history_2, loss_2, accuracy_2),
    ('4d', keras_model_3, history_3, loss_3, accuracy_3),
]:
    rows.append({'model': label, 'parameters': model.count_params(),
                 'best_epoch': int(np.argmax(hist.history['val_accuracy'])+1),
                 'validation_accuracy': float(max(hist.history['val_accuracy'])),
                 'test_loss': float(test_loss), 'test_accuracy': float(test_acc)})
table = ['| Model | Parametry | Wybrana epoka | Val accuracy | Test loss | Test accuracy |',
         '|---|---|---|---|---|---|']
for r in rows:
    table.append(f"| {r['model']} | {r['parameters']} | {r['best_epoch']} | "
                 f"{r['validation_accuracy']:.2%} | {r['test_loss']:.4f} | {r['test_accuracy']:.2%} |")
display(Markdown('\n'.join(table)))
improvement = 100 * (accuracy_3 - accuracy_2)
result_text = 'uzyskał wyższą dokładność' if improvement > 0 else 'nie uzyskał wyższej dokładności'
display(Markdown(f'Model 4d **{result_text}** od 4b; różnica to '
                 f'**{improvement:+.2f} p.p.** na zbiorze testowym. '
                 'Więcej filtrów (32/64/128) i trzeci poziom splotu zwiększają '
                 'zdolność rozpoznawania złożonych wzorców. Trzy poolingi '
                 'redukują mapy do 4×4, a dropout 0.4 ogranicza przeuczenie '
                 'warstwy gęstej. Większa liczba parametrów zwiększa koszt treningu. '
                 'Wykresy treningu i walidacji pozwalają ocenić lukę generalizacji. '
                 'Zbiór testowy służy wyłącznie końcowemu porównaniu.'))

import json
report = {'seed': SEED, 'epochs': epochs, 'batch_size': batch_size,
          'learning_rate': learning_rate, 'train_size': len(x_train),
          'validation_size': len(x_val), 'test_size': len(x_test), 'models': rows,
          'histories': {name: hist.history for name, hist in
                        [('baseline', history), ('model_4b', history_2), ('model_4d', history_3)]}}
(ARTIFACTS / 'training_results.json').write_text(
    json.dumps(report, indent=2), encoding='utf-8')
assert all(0 < r['test_accuracy'] < 1 and np.isfinite(r['test_loss']) for r in rows)
print('Zapisano historie treningów, metryki i wagi najlepszych epok w artifacts/.')
```

```
Model: "model_4d"
```

Layer (type)	Output Shape	Param #
conv2d_15 (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d_4 (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_16 (Conv2D)	(None, 16, 16, 64)	18,496
max_pooling2d_5 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_17 (Conv2D)	(None, 8, 8, 128)	73,856
max_pooling2d_6 (MaxPooling2D)	(None, 4, 4, 128)	0
flatten_2 (Flatten)	(None, 2048)	0
dense_4 (Dense)	(None, 128)	262,272
dropout (Dropout)	(None, 128)	0
dense_5 (Dense)	(None, 10)	1,290

Total params: 356,810 (1.36 MB)

Trainable params: 356,810 (1.36 MB)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/15
```

```
C:\Users\igor\source\multimodal-ai\deep-learning\lab1\.venv\Lib\site-packages\keras\src\trainers\epoch_iterator.py:74: UserWarning: `shuffle=True` was passed, but will be ignored since the data `x` was provided as a tf.data.Dataset. The Dataset is expected to already be shuffled (via `.shuffle(buffer_size)`).
  self.data_adapter = data_adapters.get_data_adapter(
```

```
704/704 - 11s - 16ms/step - accuracy: 0.4002 - loss: 1.6295 - val_accuracy: 0.5512 - val_loss: 1.2527
```

```
Epoch 2/15
```

```
704/704 - 10s - 14ms/step - accuracy: 0.5592 - loss: 1.2379 - val_accuracy: 0.6390 - val_loss: 1.0451
```

```
Epoch 3/15
```

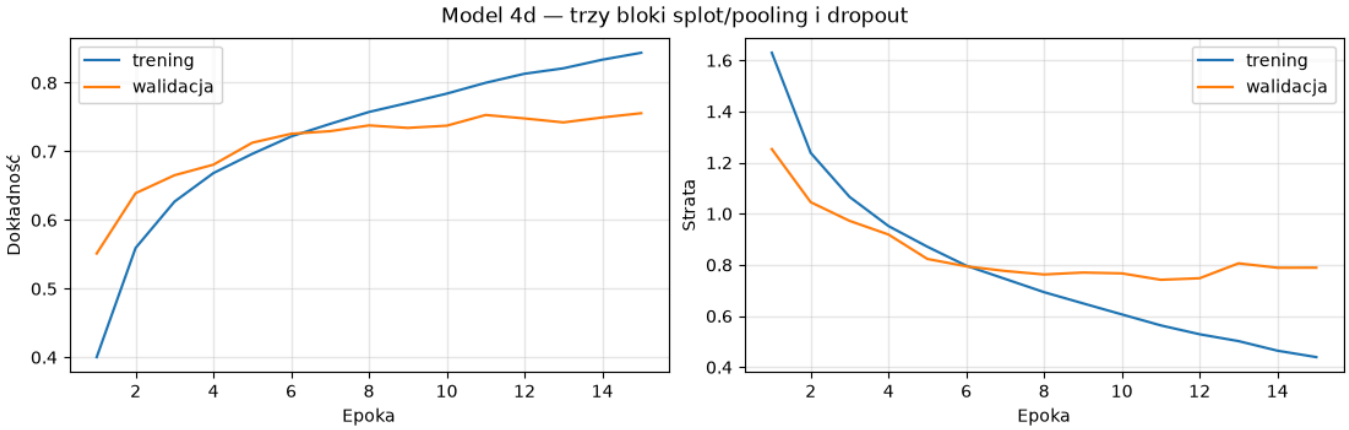
```
704/704 - 10s - 14ms/step - accuracy: 0.6268 - loss: 1.0657 - val_accuracy: 0.6652 - val_loss: 0.9721
```

```
Epoch 4/15
```

```
704/704 - 16s - 23ms/step - accuracy: 0.6683 - loss: 0.9518 - val_accuracy: 0.6806 - val_loss: 0.9187
```

```
Epoch 5/15
704/704 - 9s - 13ms/step - accuracy: 0.6964 - loss: 0.8710 - val_accuracy: 0.7126 - val_loss: 0.8232
Epoch 6/15
704/704 - 11s - 15ms/step - accuracy: 0.7217 - loss: 0.7968 - val_accuracy: 0.7256 - val_loss: 0.7943
Epoch 7/15
704/704 - 10s - 15ms/step - accuracy: 0.7400 - loss: 0.7451 - val_accuracy: 0.7294 - val_loss: 0.7761
Epoch 8/15
704/704 - 9s - 13ms/step - accuracy: 0.7574 - loss: 0.6933 - val_accuracy: 0.7378 - val_loss: 0.7625
Epoch 9/15
704/704 - 10s - 14ms/step - accuracy: 0.7706 - loss: 0.6495 - val_accuracy: 0.7342 - val_loss: 0.7700
Epoch 10/15
704/704 - 31s - 44ms/step - accuracy: 0.7841 - loss: 0.6063 - val_accuracy: 0.7374 - val_loss: 0.7668
Epoch 11/15
704/704 - 9s - 13ms/step - accuracy: 0.7999 - loss: 0.5634 - val_accuracy: 0.7530 - val_loss: 0.7417
Epoch 12/15
704/704 - 10s - 14ms/step - accuracy: 0.8131 - loss: 0.5287 - val_accuracy: 0.7480 - val_loss: 0.7478
Epoch 13/15
704/704 - 10s - 14ms/step - accuracy: 0.8210 - loss: 0.5019 - val_accuracy: 0.7422 - val_loss: 0.8060
Epoch 14/15
704/704 - 11s - 15ms/step - accuracy: 0.8336 - loss: 0.4644 - val_accuracy: 0.7494 - val_loss: 0.7889
Epoch 15/15
```

```
704/704 - 16s - 23ms/step - accuracy: 0.8437 - loss: 0.4393 - val_accuracy: 0.7556 - val_loss: 0.7890
```



Model	Parametry	Wybrana epoka	Val accuracy	Test loss	Test accuracy
Bazowy	103058	14	46.74%	1.5954	45.57%
4b	105688	14	67.16%	0.9797	66.02%
4d	356810	15	75.56%	0.8145	74.87%

Model 4d **uzyskał wyższą dokładność** od 4b; różnica to **+8.85 p.p.** na zbiorze testowym. Więcej filtrów (32/64/128) i trzeci poziom splotu zwiększają zdolność rozpoznawania złożonych wzorców. Trzy poolingi redukują mapy do 4×4, a dropout 0.4 ogranicza przeuczenie warstwy gęstej. Większa liczba parametrów zwiększa koszt treningu. Wykresy treningu i walidacji pozwalają ocenić lukę generalizacji. Zbiór testowy służy wyłącznie końcowemu porównaniu.

```
Zapisano historie treningów, metryki i wagi najlepszych epok w artifacts/.
```

Interpretacja uzyskanych wyników i przeuczenia

Wyniki dotyczą wag wybranych według dokładności walidacyjnej, a nie ostatniej epoki każdego modelu.

- **Bazowy**, epoka 14: trening 59.24%, walidacja 46.74%; luka 12.50 p.p. Test: **45.57%**.
- **4b**, epoka 14: trening 72.37%, walidacja 67.16%; luka 5.21 p.p. Test: **66.02%**.
- **4d**, epoka 15: trening 84.37%, walidacja 75.56%; luka 8.81 p.p. Test: **74.87%**.

Model bazowy wyraźnie przeucza się: dokładność treningowa nadal rośnie, podczas gdy walidacyjna stabilizuje się na dużo niższym poziomie. Model 4b poprawia dokładność testową o 20.45 p.p. i ma mniejszą lukę trening–walidacja w wybranej epoce. Model 4d poprawia wynik 4b o 8.85 p.p., ale pod koniec także się przeucza: strata treningowa maleje, a walidacyjna po początkowym spadku przestaje maleć. Dropout ogranicza ten problem, lecz go nie eliminuje. Dalsze ulepszenia można sprawdzać na walidacji, np. augmentację lub silniejszą regularyzację; tutaj oceniono jedną ustaloną architekturę 4d bez przeszukiwania na teście.

4. Forma i zawartość sprawozdania

Sprawozdanie powinno zawierać kopie ekranu stworzonych kodów i wyników ich działania dla zadań, które w instrukcji zostały oznaczone kolorem zielonym oraz stosowne komentarze, jeśli zadanie tego wymaga. Dokument powinien zostać przesłany na serwer wskazany przez prowadzącego ćwiczenie w formacie PDF.

Dodatki

Załącznik 1: Instrukcja wraz z wykonywalnymi kodami programów w formie pliku Jupyter Notebook

Załącznik 2: Obraz **lab1_1.jpg**

Fundusze Europejskie Polska Cyfrowa
Rzeczpospolita Polska
Unia Europejska Europejski Fundusz Rozwoju Regionalnego